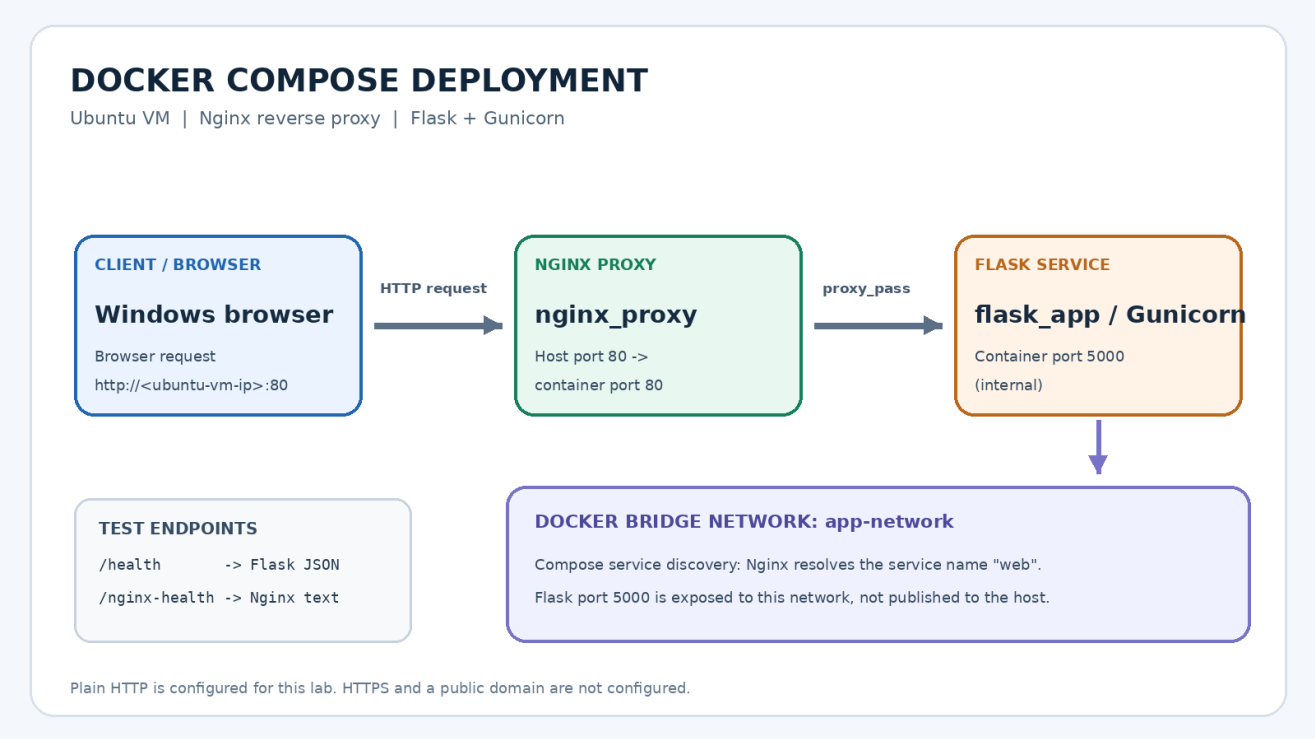
DOCKER COMPOSE DEPLOYMENT

Flask Application Behind an Nginx Reverse Proxy on Ubuntu



Technical overview, deployment procedure, validation, operations, and security scope

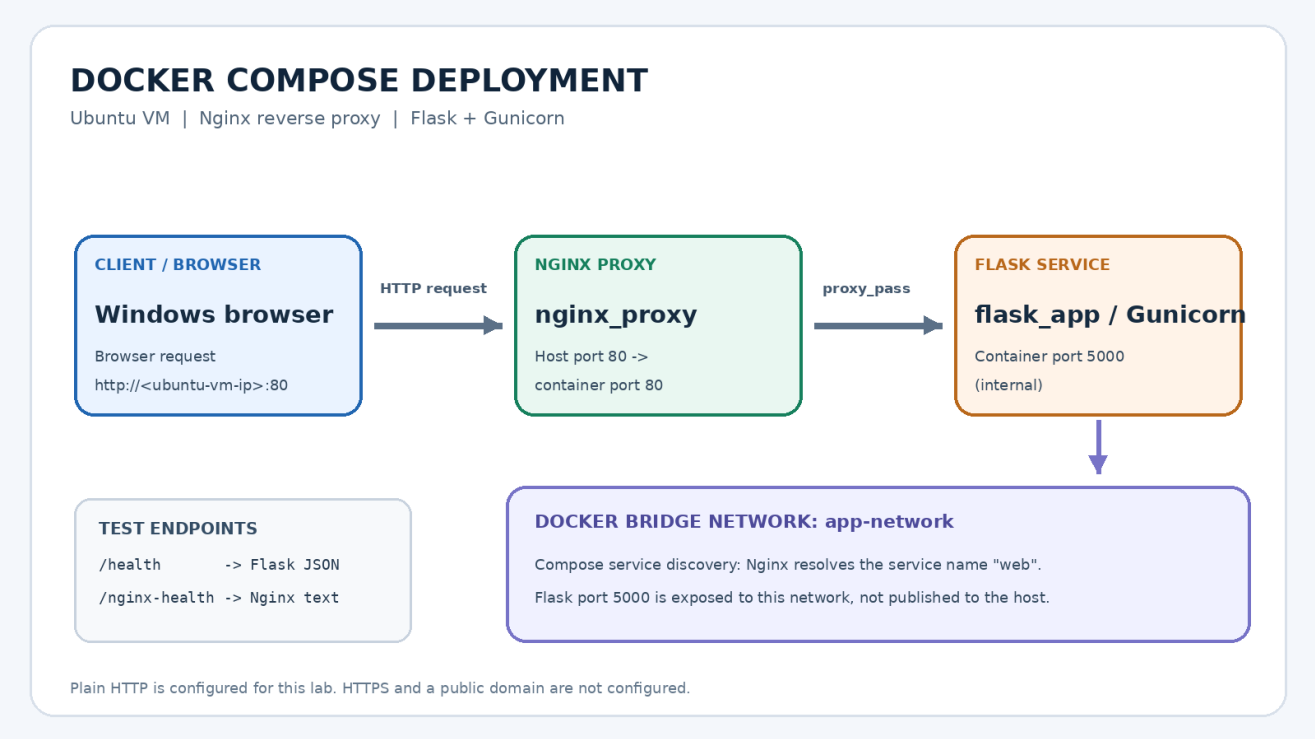
Project state: The supplied session evidence shows the homepage in a browser. In the guided build, both Compose services reached the Up state and the Nginx configuration passed nginx -t. Run the endpoint checks in this guide to capture the remaining validation evidence.

1. Project Overview

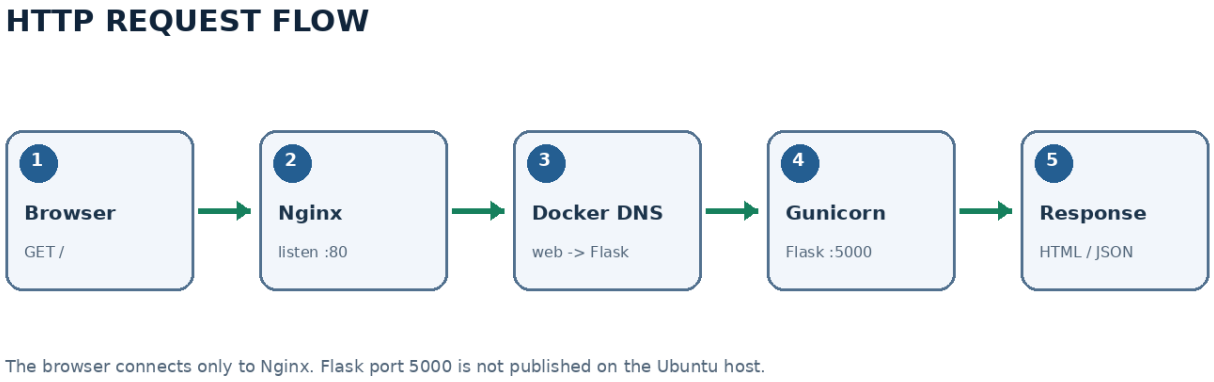
This lab packages a Python Flask application in a Docker image and deploys it on Ubuntu with Docker Compose. Nginx provides the HTTP entry point and reverse-proxies requests to the Flask service over a user-defined Docker bridge network.

The deployment separates the publicly published web entry point from the internal application port. The Ubuntu host publishes TCP port 80 for Nginx. Flask/Gunicorn listens on port 5000 inside its container and is available to the Nginx service through the Compose network.

2. Architecture and Request Flow



Request flow



The Nginx upstream is `web:5000`. The name `web` is the Compose service name, which Docker Compose resolves on the shared network. It is not the container name `flask_app`.

3. Components and Ports

Component	Role	Exposure
Ubuntu VM	Runs Docker Engine and Docker Compose.	Reachable lab VM address; current IP is learned with hostname -I.
Nginx / nginx_proxy	Reverse proxy and web entry point.	Host TCP 80 mapped to container TCP 80.
Flask / flask_app	Python application served by Gunicorn.	Container TCP 5000 on app-network only; not mapped to host.
app-network	User-defined bridge network and service discovery.	Internal Docker network.

4. Prerequisites and Lab Versions

Required: an Ubuntu VM or server, Docker Engine, Docker Compose v2, free host TCP port 80, and network reachability from the browser machine to the VM. The lab build recorded Ubuntu 26.04.1 LTS (Resolute), Docker 29.1.3, and Docker Compose 2.40.3. Package versions can vary by Ubuntu release.

```
sudo apt update
sudo apt install -y docker.io docker-compose-v2
sudo systemctl enable --now docker
docker --version
docker compose version
```

5. Deployment Procedure

Run commands from the project directory on Ubuntu: `~/projects/docker-compose-nginx-ubuntu`.

```
cd ~/projects/docker-compose-nginx-ubuntu
sudo docker compose config
sudo docker compose build web
sudo docker compose up -d
sudo docker compose ps
```

Expected running services

Service	Expected state	Published port
flask_app	Up	None. Port 5000 is internal.
nginx_proxy	Up	0.0.0.0:80 -> container 80 (IPv4/IPv6 mapping may appear).

6. Application Endpoints

Path	Handled by	Expected result
/	Flask via Nginx	HTTP 200 and a simple HTML welcome page.
/health	Flask via Nginx	HTTP 200 JSON with status, hostname, and UTC timestamp.
/nginx-health	Nginx directly	HTTP 200 text: Nginx is healthy.

```
curl -i http://localhost/
curl -i http://localhost/health
curl -i http://localhost/nginx-health
```

Access from Windows or another machine

```
hostname -I
```

Use the current Ubuntu VM address in a browser: `http://<UBUNTU-VM-IP>/`. The original session evidence showed `192.168.109.129`; VMware NAT/DHCP addresses can change. Do not assume this address is permanent.

7. Operations

```
sudo docker compose ps
sudo docker compose logs --tail=50 nginx web
sudo docker compose logs -f nginx web
sudo docker compose restart
sudo docker compose down
sudo docker compose up -d
```

`docker compose down` removes the containers and project network; it does not remove source files. There is no database or persistent data volume in the current project.

8. Troubleshooting

Port 80 already in use: identify the process with `sudo ss -ltnp | grep :80`. The lab encountered Apache2 using port 80. If Apache is not needed for another workload, stop it with `sudo systemctl stop apache2`. Do not disable or uninstall it without checking dependencies.

Nginx reports an unexpected closing brace: inspect `cat -n nginx/default.conf` and ensure braces are balanced. Test the mounted file with `sudo docker compose run --rm nginx nginx -t`.

Nginx cannot resolve upstream web: ensure `web` and `nginx` services are both attached to `app-network`. The Nginx upstream must be `web:5000`, using the Compose service name. Recreate a stale stack with `sudo docker compose down` followed by `sudo docker compose up -d`.

Read-only default.conf warning: the entrypoint script may report that it cannot modify the mounted config because the volume is read-only. This is expected for the `:ro` mount. Read subsequent log lines to identify any actual fatal error.

9. Security Scope and Limitations

The application process runs as a non-root user in the image. Flask port 5000 is not published to the host. Nginx configuration is mounted read-only. The deployment currently uses plain HTTP on port 80.

A public domain, TLS certificate, HTTPS listener, production secrets management, persistent database, automated deployment pipeline, image scanning, and production monitoring are not included. Treat this as a learning/lab deployment, not a fully production-hardened service.

10. Evidence Checklist

Evidence item	What it demonstrates
Browser homepage	HTTP access through Nginx to Flask.
/health response	Flask application endpoint returns JSON.
/nginx-health response	Nginx responds directly.
docker compose ps	Both services remain in Up state.
nginx -t output	Nginx configuration syntax is valid.

Capture screenshots from the real running environment. The architecture graphic is explanatory artwork and must not be presented as proof of a live test.